

山东大学



作业三：对称密码算法的软件实现与优化

AES、SM4、GIFT、TWINE 及 CTR/GCM/XTS 工作模式优化实现

成员姓名	学号	班级	主要分工
马悦曦	202300201057	网安 2 班	总体架构、AES 全部优化、GCM/GHASH、GIFT-64 基础与 SP-table、系统集成
周思琪	202300450129	密码 1 班	PCLMULQDQ、SM4 全部优化、TWINE-80/128 实现、性能数据采集
闫雨彤	202322161173	密码 1 班	CTR 与 XTS 实现及联调、GIFT/TWINE 标准向量、正确性测试
孙世瑾	202300210010	密码 1 班	AVX2/VAES 并行优化、轻量密码性能测试、README、LaTeX 排版和提交材料整理

2026 年 7 月 31 日

目录

1 实验概览	III
1.1 实验题目	III
1.2 实验目标	III
2 实验环境与工程结构	III
2.1 实验环境	III
2.2 工程目录结构	IV
3 对称密码与优化原理	V
3.1 AES-128 算法原理	V
3.2 SM4 算法原理	V
3.3 GIFT-64 算法原理	V
3.4 TWINE 算法原理	VI
3.5 T-table 优化原理	VI
3.6 Shuffle 与 SIMD 优化原理	VII
3.7 AES-NI、VAES 与 AVX2	VII
3.8 CTR 工作模式	VIII
3.9 GCM 工作模式	VIII
3.10 XTS 工作模式	VIII
4 实验代码实现与分析	IX
4.1 AES-128 密钥扩展	IX
4.2 AES 基础加密轮	IX
4.3 AES T-table 生成	X
4.4 AES T-table 轮函数	X
4.5 Shuffle 实现 ShiftRows	X
4.6 SIMD 并行计算 MixColumns	XI
4.7 AES-NI 加密	XI
4.8 VAES 八分组并行	XI
4.9 AES-CTR 串行核心	XII
4.10 AES-CTR 的 VAES 八路主循环	XII
4.11 GHASH 基础有限域乘法	XIII
4.12 PCLMULQDQ 优化 GHASH	XIII
4.13 GCM 认证数据组织	XIV
4.14 GCM 标签验证与失败处理	XIV
4.15 XTS tweak 更新	XIV
4.16 XTS 分组加密	XV
4.17 SM4 非线性与线性变换	XV
4.18 SM4 基础 32 轮	XV
4.19 SM4 T-table 构造	XVI
4.20 SM4 AVX2 八路并行查表	XVI
4.21 SM4-CTR 八路处理	XVI

4.22	GIFT-64 基础轮函数	XVII
4.23	GIFT-64 SP-table 构造	XVII
4.24	GIFT-64 AVX2 八分组并行	XVII
4.25	TWINE 基础 F 层与轮置换	XVIII
4.26	TWINE-80 与 TWINE-128 密钥扩展	XVIII
4.27	TWINE AVX2 寄存器内 Shuffle	XIX
4.28	统一编译与运行脚本	XIX
5	正确性验证与实验步骤	XIX
5.1	标准测试向量	XIX
5.2	鲁棒性测试	XX
5.3	实际运行步骤	XX
5.4	正确性测试结果	XXI
6	性能结果与分析	XXI
6.1	性能测试方法	XXI
6.2	AES 分组密码性能	XXII
6.3	AES-CTR 性能	XXII
6.4	AES-GCM 性能	XXIII
6.5	AES-XTS 性能	XXIII
6.6	SM4 与 SM4-CTR 性能	XXIV
6.7	GIFT 与 TWINE 轻量密码性能	XXIV
6.8	总体性能图	XXV
6.9	结果有效性分析	XXVI
7	安全性与工程讨论	XXVI
7.1	T-table 缓存侧信道	XXVI
7.2	CTR 的 Nonce/计数器复用风险	XXVI
7.3	GCM 的 IV 唯一性与认证顺序	XXVI
7.4	XTS 的安全边界	XXVII
7.5	性能测试的公平性	XXVII
8	实验中遇到的问题与解决方法	XXVII
8.1	脚本执行权限问题	XXVII
8.2	WSL 绘图后端问题	XXVII
8.3	优化正确性问题	XXVII
8.4	尾分组与原地操作	XXVII
8.5	轻量密码的半字节与比特映射	XXVIII
9	实验总结	XXVIII

1 实验概览

1.1 实验题目

本次作业的题目为“对称密码算法的软件实现”。实验从分组密码的基础软件实现出发，对 AES-128、SM4、GIFT-64 与 TWINE 进行多层次优化，至少覆盖 T-table、shuffle 以及现代指令集两类方法；在此基础上进一步实现并优化 CTR、GCM 与 XTS 工作模式。实验工程面向 x86-64 平台，既保留能够清楚反映算法步骤的基础版本，也给出面向 Intel Core i7-13700H 的 AES-NI、VAES、AVX2 和 PCLMULQDQ 优化版本。新增的轻量密码部分中，GIFT-64 实现基础轮函数、SP-table 与 AVX2 八分组并行，TWINE 同时实现 80 bit 和 128 bit 密钥版本，并使用 VPSHUFb 完成向量寄存器内的 S 盒与置换操作。

本实验不是简单调用第三方密码库，而是独立实现密钥扩展、轮函数、工作模式、认证计算、尾分组处理和测试框架。所有优化版本均与基础版本及公开标准测试向量进行对照，确保性能优化没有破坏算法语义。

1.2 实验目标

本实验的主要目标如下：

1. 掌握 AES-128、SM4、GIFT-64 与 TWINE 的状态表示、密钥扩展、轮函数和加解密流程；
2. 理解 T-table 如何将 S 盒替换和线性变换合并为查表与异或操作；
3. 理解 shuffle 指令如何完成固定字节置换，并利用 SIMD 并行计算 MixColumns；
4. 掌握 AES-NI 中 AESENC、AESENC_LAST、AESDEC 等专用指令的使用；
5. 使用 VAES 和 AVX2 实现多分组并行，提高批量数据吞吐率；
6. 掌握 GIFT-64 的 SubCells、PermBits 和轮密钥注入，以及 TWINE 的 4 bit F 函数、置换层和 80/128 bit 密钥扩展；
7. 实现 CTR、GCM 和 XTS 工作模式，正确处理计数器、认证标签、tweak 和非整分组数据；
8. 使用 PCLMULQDQ 加速 GCM 中的 GHASH 有限域乘法；
9. 使用 FIPS、NIST 与国标测试向量验证正确性，并在真实 CPU 上测量吞吐率和加速比；
10. 分析不同优化方法的适用条件、性能瓶颈以及缓存侧信道等安全问题。

2 实验环境与工程结构

2.1 实验环境

实验在 WSL Ubuntu 环境中完成，处理器为 13th Gen Intel Core i7-13700H。该处理器在 WSL 中暴露的关键指令集包括 SSSE3、AES、AVX2、VAES、PCLMULQDQ 和 VPCLMULQDQ，适合比较通用软件优化与密码专用指令优化。

表 1: 实验软硬件环境

项目	配置
操作系统	Windows Subsystem for Linux, Ubuntu, x86-64
CPU	13th Gen Intel Core i7-13700H, 20 个逻辑处理器
编译器	GCC, C11/gnu11
编译优化	-O3 -Wall -Wextra -Wpedantic
指令参数	-mssse3 -maes -mavx2 -mvaes -mpclmul
绘图环境	Python 3, Matplotlib, Agg 无图形后端
计量指标	MiB/s, 使用单调时钟统计批量数据吞吐率

2.2 工程目录结构

Listing 1: 实验工程目录结构

```
1 SymmetricCryptoOptimization_AllFour/
2 |-- include/
3 |   |-- aes.h, aes_ctr.h, aes_gcm.h, aes_xts.h
4 |   |-- sm4.h, sm4_ctr.h
5 |   |-- gift64.h, twine.h
6 |-- src/
7 |   |-- aes_common.c, aes_basic.c, aes_ttable.c
8 |   |-- aes_shuffle.c, aes_aesni.c, aes_vaes.c
9 |   |-- aes_ctr.c, aes_gcm.c, aes_xts.c
10 |   |-- sm4_common.c, sm4_basic.c, sm4_ttable.c
11 |   |-- sm4_avx2.c, sm4_ctr.c
12 |   |-- gift64.c, twine.c
13 |-- tests/
14 |   |-- test_*_bench.c
15 |   |-- test_lightweight_bench.c
16 |-- scripts/
17 |   |-- merge_results.py
18 |   |-- plot_results.py
19 |-- results/
20 |   |-- *_benchmark.csv, lightweight_benchmark.csv
21 |   |-- performance.png
22 |-- Makefile
23 |-- run_all.sh
```

工程将算法接口放在 `include` 目录, 将具体实现放在 `src` 目录。测试程序同时承担正确性验证和性能测量功能。`run_all.sh` 依次完成清理、编译、快速测试、正式 benchmark、结果合并和绘图, 保证实验过程可复现。

3 对称密码与优化原理

3.1 AES-128 算法原理

AES 是典型的替换-置换网络。AES-128 的分组长度和密钥长度均为 128 bit，状态可表示为 4×4 字节矩阵，共执行 10 轮。设输入状态为 S ，轮密钥为 K_r ，整体过程为：

1. 初始轮： $S \leftarrow S \oplus K_0$ ；
2. 第 1 至第 9 轮：依次执行 SubBytes、ShiftRows、MixColumns 和 AddRoundKey；
3. 第 10 轮：执行 SubBytes、ShiftRows 和 AddRoundKey，不执行 MixColumns。

SubBytes 通过固定 S 盒完成逐字节非线性变换，是 AES 抵抗线性分析与差分分析的核心。ShiftRows 对状态矩阵第 r 行循环左移 r 个字节，使同一列中的字节扩散到不同列。MixColumns 将每一列视为 $GF(2^8)$ 上的四维向量，与固定矩阵相乘：

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}.$$

有限域模多项式为 $x^8 + x^4 + x^3 + x + 1$ ，因此乘 2 可通过左移并按最高位条件异或 0x1b 实现。AddRoundKey 则将状态与轮密钥逐字节异或。

AES-128 密钥扩展由原始 16 字节密钥生成 11 组轮密钥，共 176 字节。每生成 16 字节时，对前一个字执行 RotWord、SubWord 并异或轮常量 Rcon，其余字通过与前 16 字节异或递推获得。

3.2 SM4 算法原理

SM4 同样使用 128 bit 分组和 128 bit 密钥，但内部以 4 个 32 bit 字表示状态，共执行 32 轮非线性迭代。设第 i 轮输入为 $X_i, X_{i+1}, X_{i+2}, X_{i+3}$ ，轮密钥为 rk_i ，则

$$X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i).$$

复合变换 $T = L \circ \tau$ 。其中 τ 将 32 bit 输入拆成 4 个字节，每个字节经过 SM4 S 盒；线性变换为

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

32 轮结束后进行反序输出，即 $Y = (X_{35}, X_{34}, X_{33}, X_{32})$ 。SM4 解密不需要构造独立的逆轮函数，只需将 32 个轮密钥逆序使用。

SM4 密钥扩展先将主密钥的四个字与系统参数 FK 异或，再迭代生成 32 个轮密钥。密钥扩展的非线性变换与加密相同，但线性部分改为

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

3.3 GIFT-64 算法原理

GIFT 是一类面向受限设备的轻量级分组密码。本实验实现 GIFT-64：分组长度为 64 bit，密钥长度为 128 bit，共执行 28 轮。64 bit 状态被划分为 16 个 4 bit 半字节

$$S = (s_0, s_1, \dots, s_{15}),$$

每一轮依次执行 SubCells、PermBits 和 AddRoundKey:

$$S^{(r+1)} = \text{AddRoundKey}(\text{PermBits}(\text{SubCells}(S^{(r)})), K_r, RC_r).$$

SubCells 对 16 个半字节分别使用同一 4 bit S 盒。与 AES 的 8 bit S 盒相比, 4 bit S 盒规模更小, 适合低面积硬件和轻量软件实现。PermBits 不是按字节或半字节整体移动, 而是把 64 个状态比特按照固定映射 $P(i)$ 重新放置。该置换能够把每个 S 盒输出的 4 个比特扩散到不同的后续 S 盒输入中, 使非线性影响逐轮扩展到整个状态。

AddRoundKey 只在指定比特位置注入轮密钥, 同时注入 6 bit 轮常量 RC_r 和固定常量位。实现中预先把一轮需要异或的所有密钥位和常量位组合为 64 bit 的 `round_mask`, 因此实际轮函数可直接写成

$$S \leftarrow \text{PermBits}(\text{SubCells}(S)) \oplus M_r.$$

密钥状态以 32 个半字节保存, 每轮通过半字节重排和局部旋转更新。解密过程按相反顺序执行轮掩码异或、逆比特置换和逆 S 盒, 因此能够与加密共享同一轮密钥上下文。

3.4 TWINE 算法原理

TWINE 同样采用 64 bit 分组, 支持 80 bit 和 128 bit 两种密钥长度, 共执行 36 轮。状态被表示为 16 个 4 bit 半字节 $x_0, \dots, x_{15}$ 。每轮包含 8 个并行 F 函数, 偶数位置作为非线性函数输入, 结果异或到相邻奇数位置:

$$x_{2j+1} \leftarrow x_{2j+1} \oplus S(x_{2j} \oplus rk_j), \quad 0 \leq j < 8.$$

前 35 轮在 F 函数之后执行固定半字节置换 π , 最后一轮只执行 F 函数而不再置换。由于一轮中 8 个 F 函数之间互不依赖, TWINE 天然适合在 SIMD 寄存器内并行处理; 而跨分组并行又能够把这种并行度进一步扩大。

TWINE-80 和 TWINE-128 使用相同的数据轮函数, 但密钥状态长度及每轮抽取轮密钥的位置不同。80 bit 密钥被拆为 20 个半字节, 128 bit 密钥被拆为 32 个半字节; 两种密钥扩展都包含 S 盒更新、轮常量注入和 4 半字节粒度的循环更新。实现阶段在初始化时预先生成 36 组、每组 8 个半字节的轮密钥, 因此正式加密过程中不再重复执行密钥调度。也正因为数据轮函数完全相同, 两种密钥长度的批量加密性能应较为接近。

3.5 T-table 优化原理

基础实现会在每轮分别执行 S 盒访问、字节置换和有限域乘法。T-table 的思想是将若干确定变换预先合并, 使运行时只执行查表与异或。

以 AES 为例, 某个输入字节 x 经过 S 盒后得到 $s = S(x)$ 。根据该字节在列中的位置, 可预先计算

$$T_0[x] = (2s, s, s, 3s),$$

其余 T_1, T_2, T_3 由字节旋转得到。这样一系列的 SubBytes、ShiftRows 和 MixColumns 可写成四次查表与异或:

$$t_0 = T_0[s_{0,0}] \oplus T_1[s_{1,1}] \oplus T_2[s_{2,2}] \oplus T_3[s_{3,3}] \oplus rk.$$

AES 末轮没有 MixColumns, 因此仍单独执行 S 盒与字节选择。

SM4 也可以将 τ 与 L 合并。对输入 32 bit 字的四个字节分别建立 T_0 至 T_3 , 有

$$T(x) = T_0[x_{31:24}] \oplus T_1[x_{23:16}] \oplus T_2[x_{15:8}] \oplus T_3[x_{7:0}].$$

GIFT-64 的基础实现需要对每个半字节执行 S 盒，再逐比特执行 PermBits。由于 S 盒输出的每一位在置换后的目标位置固定，可以为 16 个半字节位置分别建立 16 项 SP-table:

$$SP_i[x] = \text{PermBits}_i(S(x)).$$

一轮的 SubCells 与 PermBits 便可合并为

$$Y = \bigoplus_{i=0}^{15} SP_i[s_i], \quad S' = Y \oplus M_r.$$

这里每张表只含 16 个 64 bit 表项，总体积较小，却能消除基础版本中每轮 64 次逐比特提取与写回，是 GIFT 软件实现中最明显的优化来源。

T-table 显著减少了有限域运算、循环移位或逐比特置换，但访问地址依赖秘密状态，会在共享缓存环境中暴露访存模式。因此它适合作为课程中的软件性能对比方法，在生产系统中应优先使用常数时间的专用指令、bitslicing 或寄存器内 shuffle 实现。

3.6 Shuffle 与 SIMD 优化原理

Shuffle 指令用于在向量寄存器内按照掩码重新排列字节。AES 的 ShiftRows 是固定置换，因此可以构造常量掩码

$$(0, 5, 10, 15, 4, 9, 14, 3, 8, 13, 2, 7, 12, 1, 6, 11),$$

并通过 PSHUFB 一次完成 16 字节重排。MixColumns 中各列具有相同计算结构，可把 16 个字节同时放入 128 bit 寄存器，使用并行移位、掩码和异或计算所有字节的 `xtime`。

本实验的 Shuffle 版本仍采用普通 S 盒数组，因此它主要消除了 ShiftRows 和部分 MixColumns 的标量开销，并未消除秘密相关查表。实验结果可用来观察“只优化字节置换”与“直接使用 AES 专用轮指令”之间的性能差异。

TWINE 的 4 bit S 盒只有 16 项，可以把 S 盒完整复制到 AVX2 寄存器的两个 128 bit lane 中，并使用 VPSHUFB 把每个半字节直接作为索引。固定置换 π 也能通过另一条 VPSHUFB 完成，因此 S 盒与置换都不需要秘密相关的内存地址。实现一次并行处理 8 个 TWINE 分组，属于典型的“寄存器内查表 + 跨分组并行”组合。

GIFT-64 的 SP-table 表项为 64 bit，不能直接使用字节 shuffle 完成查表，因此 AVX2 版本把 8 个分组拆为两组 4 个 64 bit lane，并使用 VPGATHERQ 按半字节索引读取 SP-table。该方法保留了 SP-table 合并变换的优势，同时并行推进 8 条独立状态链。

3.7 AES-NI、VAES 与 AVX2

AES-NI 提供与 AES 轮函数直接对应的硬件指令。AESENC 完成 SubBytes、ShiftRows、MixColumns 和 AddRoundKey，AESENCLAST 完成不含 MixColumns 的末轮；解密对应 AESDEC 与 AESDECLAST，AESIMC 用于转换解密轮密钥。

AES-NI 每条指令处理一个 128 bit 分组，适合降低单分组延迟。VAES 将相同轮语义扩展到 256 bit 或 512 bit 向量。实验使用 256 bit VAES 寄存器，每个寄存器容纳两个 AES 分组，同时保持四条独立状态链，因此一次函数调用处理 8 个分组。多条独立依赖链可以隐藏轮指令延迟，使处理器执行单元保持饱和。

SM4 没有在本机上直接使用专用 SM4 指令，因此采用 AVX2 八路并行。把 8 个分组中相同位置的 32 bit 状态字放入 8 个 lane，用向量异或完成轮输入组合，再利用 VPGATHERDD 从四张 T 表并行取值。该布局属于“跨分组并行”，适用于 CTR 等分组之间相互独立的场景。

轻量密码也采用跨分组并行。GIFT-64 以 64 bit lane 承载一个完整状态，两个 256 bit 寄存器共处理 8 个分组；TWINE 则把每个分组展开为 16 个半字节，四个 256 bit 寄存器各容纳两个分组。两种布局都让同一轮中的 8 个分组互不依赖，从而隐藏查表、shuffle 和轮密钥注入的执行延迟。

3.8 CTR 工作模式

CTR 把分组密码转化为流密码。设初始计数器为 CTR_0 ，则

$$C_i = P_i \oplus E_K(CTR_i), \quad CTR_{i+1} = CTR_i + 1.$$

解密公式完全相同，因为再次异或同一密钥流即可恢复明文。CTR 不要求输入长度是 16 字节整数倍，末尾只需使用密钥流的前若干字节。

CTR 中不同计数器分组没有数据依赖，因此非常适合 VAES 和 AVX2 多分组并行。本实验一次构造 8 个连续计数器，加密得到 128 字节密钥流，再与 128 字节输入异或；不足 128 字节的尾部回退到单分组实现。计数器按 128 bit 大端整数递增，并测试了低位溢出后的进位行为。

3.9 GCM 工作模式

GCM 同时提供机密性和完整性。其加密部分基于 CTR，认证部分由 GHASH 完成。令

$$H = E_K(0^{128}),$$

GHASH 把附加认证数据 AAD、密文和长度块依次吸收，迭代关系为

$$X_i = (X_{i-1} \oplus B_i) \cdot H,$$

乘法位于 $GF(2^{128})$ ，模多项式为

$$x^{128} + x^7 + x^2 + x + 1.$$

最终认证标签为

$$T = E_K(J_0) \oplus GHASH_H(A, C),$$

其中 96 bit IV 的 J_0 可直接构造为 $IV \parallel 0^{31} \parallel 1$ ；一般长度 IV 则先经过 GHASH。

基础 GHASH 逐比特执行条件异或和移位约简，计算量较大。PCLMULQDQ 能够在硬件中完成 64 bit 多项式的无进位乘法，通过四个部分积组合为 256 bit 结果，再模不可约多项式约简到 128 bit。GCM 中 GHASH 存在迭代依赖，因此即使 CTR 部分使用 VAES 八路并行，总吞吐率仍会受到认证链限制。

3.10 XTS 工作模式

XTS 主要用于磁盘和存储数据单元加密，使用两个 AES 密钥。第二个密钥生成初始 tweak：

$$T_0 = E_{K_2}(I),$$

其中 I 是数据单元编号或外部 tweak 输入。第 i 个分组加密为

$$C_i = E_{K_1}(P_i \oplus T_i) \oplus T_i, \quad T_{i+1} = \alpha T_i.$$

乘 α 在 $GF(2^{128})$ 中实现为左移一位；若最高位产生进位，则最低字节异或约简常量 0x87。

XTS 不采用普通填充，因为存储加密要求密文长度与明文长度相同。对最后一个不完整分组使用 ciphertext stealing：先加密倒数第二个完整分组，从其密文中“窃取”部分字节作为尾部密文，再重新组合并加密完整块。本实验实现基础、AES-NI 和 VAES 八路版本，并测试完整分组与 CTS 尾部。

4 实验代码实现与分析

本节按照“短代码片段 + 紧随其后的解释”展示关键实现。为避免报告中堆积大段源码，只保留能够反映算法结构或优化思想的核心部分，完整代码见工程 `src` 目录。

4.1 AES-128 密钥扩展

Listing 2: AES-128 轮密钥递推

```
1 while (generated < AES128_ROUND_KEYS_SIZE) {
2     memcpy(temp, ctx->round_keys + generated - 4, 4);
3
4     if ((generated % 16U) == 0U) {
5         const uint8_t t = temp[0];
6         temp[0] = AES_SBOX[temp[1]];
7         temp[1] = AES_SBOX[temp[2]];
8         temp[2] = AES_SBOX[temp[3]];
9         temp[3] = AES_SBOX[t];
10        temp[0] ^= RCON[rcon_i++];
11    }
12
13    for (size_t i = 0; i < 4; ++i) {
14        ctx->round_keys[generated] =
15            ctx->round_keys[generated - 16] ^ temp[i];
16        ++generated;
17    }
18 }
```

代码先复制当前已生成密钥末尾的 4 字节。当生成位置位于 16 字节边界时，执行 RotWord、SubWord 和 Rcon 异或；随后把变换结果与前 16 字节对应位置异或。循环结束后得到 176 字节轮密钥，后续所有 AES 后端共享该上下文，避免不同优化版本重复进行密钥扩展。

4.2 AES 基础加密轮

Listing 3: AES 基础版本的十轮结构

```
1 aes_add_round_key(state, ctx->round_keys);
2 for (unsigned round = 1; round < 10; ++round) {
3     aes_sub_bytes(state);
4     aes_shift_rows(state);
5     aes_mix_columns(state);
6     aes_add_round_key(state,
7         ctx->round_keys + 16 * round);
8 }
9 aes_sub_bytes(state);
10 aes_shift_rows(state);
11 aes_add_round_key(state, ctx->round_keys + 160);
```

该实现严格对应 AES 标准，适合作为正确性基线。中间 9 轮包含完整的四个步骤，末轮省略 MixColumns。基础版本的主要开销来自逐字节 S 盒、状态搬移和有限域乘法，因此吞吐率最低，但结构最直观，也便于与其他后端进行逐分组等价性验证。

4.3 AES T-table 生成

Listing 4: AES 加密 T 表的运行时初始化

```
1 for (unsigned x = 0; x < 256; ++x) {
2     const uint8_t s = AES_SBOX[x];
3     const uint32_t te =
4         ((uint32_t)aes_gmul(s, 2) << 24) |
5         ((uint32_t)s << 16) |
6         ((uint32_t)s << 8) |
7         (uint32_t)aes_gmul(s, 3);
8     TE0[x] = te;
9     TE1[x] = aes_rotr32(te, 8);
10    TE2[x] = aes_rotr32(te, 16);
11    TE3[x] = aes_rotr32(te, 24);
12 }
```

对每个可能字节 x ，先查 S 盒，再计算其在 MixColumns 第一位置上的四字节贡献。其余三张表通过 32 bit 循环右移得到。初始化只执行一次，随后每轮无需再次进行有限域乘法。工程还以逆 S 盒和逆 MixColumns 系数构造 Td0–Td3，用于 T-table 解密。

4.4 AES T-table 轮函数

Listing 5: 四次查表合并 AES 中间轮

```
1 const uint32_t t0 =
2     TE0[s0 >> 24] ^ TE1[(s1 >> 16) & 0xff] ^
3     TE2[(s2 >> 8) & 0xff] ^ TE3[s3 & 0xff] ^
4     rk[rki++];
5 const uint32_t t1 =
6     TE0[s1 >> 24] ^ TE1[(s2 >> 16) & 0xff] ^
7     TE2[(s3 >> 8) & 0xff] ^ TE3[s0 & 0xff] ^
8     rk[rki++];
```

状态被组织为 4 个 32 bit 字。索引从不同状态字的不同字节位置取得，已经隐含 ShiftRows；各 T 表项又包含 SubBytes 和 MixColumns 贡献。每个输出字只需 4 次查表、若干异或和 1 次轮密钥异或。其余 t_2, t_3 结构相同。末轮不能使用该表，因为 AES 末轮没有 MixColumns。

4.5 Shuffle 实现 ShiftRows

Listing 6: 使用 PSHUFB 完成固定字节置换

```
1 const __m128i shift_rows = _mm_setr_epi8(
2     0, 5, 10, 15,
3     4, 9, 14, 3,
```

```
4     8, 13, 2, 7,  
5     12, 1, 6, 11);  
6  
7 state = _mm_shuffle_epi8(state, shift_rows);
```

掩码中的每个值指定输出位置应选取的输入字节。由于 AES ShiftRows 的置换模式固定，运行时不需要循环和临时数组，一条 PSHUFB 即可完成全部 16 字节重排。

4.6 SIMD 并行计算 MixColumns

Listing 7: 向量化 MixColumns 的核心组合

```
1 const __m128i r1 = _mm_shuffle_epi8(s, rot1);  
2 const __m128i r2 = _mm_shuffle_epi8(s, rot2);  
3 const __m128i r3 = _mm_shuffle_epi8(s, rot3);  
4  
5 return _mm_xor_si128(  
6     _mm_xor_si128(xtime_bytes(s), xtime_bytes(r1)),  
7     _mm_xor_si128(r1, _mm_xor_si128(r2, r3)));
```

rot1、rot2 和 rot3 在每个 4 字节列内进行循环置换。根据 MixColumns 矩阵，可把输出写成 xtime(s)、xtime(rot1) 和若干旋转向量的异或。这样 16 个字节同时参与运算，避免逐列逐字节调用有限域乘法函数。

4.7 AES-NI 加密

Listing 8: AES-NI 单分组加密

```
1 __m128i s = _mm_loadu_si128((const __m128i *)in);  
2 s = _mm_xor_si128(  
3     s, _mm_loadu_si128((const __m128i *)ctx->round_keys));  
4  
5 for (unsigned round = 1; round < 10; ++round) {  
6     s = _mm_aesenc_si128(  
7         s,  
8         _mm_loadu_si128((const __m128i *)  
9             (ctx->round_keys + 16 * round)));  
10 }  
11 s = _mm_aesenclast_si128(  
12     s, _mm_loadu_si128((const __m128i *)  
13         (ctx->round_keys + 160)));
```

初始轮仍通过普通异或完成。9 个中间轮分别调用 _mm_aesenc_si128，末轮使用 _mm_aesenclast_si128。每条指令在硬件中完成完整轮变换，既消除了软件 S 盒和有限域乘法，又避免传统 T-table 的秘密相关缓存访问。

4.8 VAES 八分组并行

Listing 9: VAES 多状态链并行轮函数

```
1 __m256i s0 = _mm256_loadu_si256(  
2     (const __m256i *) (in + 0));  
3 __m256i s1 = _mm256_loadu_si256(  
4     (const __m256i *) (in + 32));  
5 __m256i s2 = _mm256_loadu_si256(  
6     (const __m256i *) (in + 64));  
7 __m256i s3 = _mm256_loadu_si256(  
8     (const __m256i *) (in + 96));  
9  
10 for (unsigned round = 1; round < 10; ++round) {  
11     rk = broadcast_rk(ctx->round_keys + 16 * round);  
12     s0 = _mm256_aesenc_epi128(s0, rk);  
13     s1 = _mm256_aesenc_epi128(s1, rk);  
14     s2 = _mm256_aesenc_epi128(s2, rk);  
15     s3 = _mm256_aesenc_epi128(s3, rk);  
16 }
```

每个 256 bit 寄存器包含两个互不干扰的 AES-128 lane，四个寄存器总计 8 个分组。轮密钥通过广播复制到两个 128 bit lane。四条独立状态链减少了连续轮之间的数据依赖对吞吐率的限制，因此该版本适合大批量、可并行分组场景。其 benchmark 反映的是批量吞吐率，不等同于单个分组延迟降低 134 倍。

4.9 AES-CTR 串行核心

Listing 10: CTR 任意长度处理逻辑

```
1 while (len != 0U) {  
2     const size_t take = len < 16U ? len : 16U;  
3     fn(ctx, counter, stream);  
4     for (size_t i = 0; i < take; ++i) {  
5         out[i] = (uint8_t)(in[i] ^ stream[i]);  
6     }  
7     in += take;  
8     out += take;  
9     len -= take;  
10    increment_be(counter);  
11 }
```

fn 是可替换的分组加密后端，因此同一 CTR 框架可以连接基础、T-table、Shuffle 或 AES-NI 实现。take 使最后一个分组可小于 16 字节，CTR 不需要填充。输入和输出指针可以相同，因此支持原地加解密。

4.10 AES-CTR 的 VAES 八路主循环

Listing 11: 一次生成八个计数器分组的密钥流

```
1 while (len >= 128U) {  
2     for (unsigned b = 0; b < 8; ++b) {
```

```
3     memcpy(counters + 16U * b, counter, 16);
4     increment_be(counter);
5 }
6 aes128_encrypt8_vaes(ctx, counters, stream);
7 for (unsigned i = 0; i < 128; ++i) {
8     out[i] = (uint8_t)(in[i] ^ stream[i]);
9 }
10 in += 128;
11 out += 128;
12 len -= 128;
13 }
```

八个连续计数器之间相互独立，可直接交给 VAES 后端。完成 128 字节主循环后，剩余数据使用 AES-NI 串行后端处理。这种“宽向量主循环 + 标量尾部”结构避免为了少量尾数据构造复杂掩码。

4.11 GHASH 基础有限域乘法

Listing 12: GHASH 逐比特乘法的核心结构

```
1 for (unsigned i = 0; i < 128; ++i) {
2     if (get_bit_be(x, i)) {
3         xor_block(z, v);
4     }
5     const unsigned lsb = v[15] & 1U;
6     shift_right_one(v);
7     if (lsb != 0U) {
8         v[0] ^= 0xe1U;
9     }
10 }
```

该算法按大端位序扫描输入 x 。当当前位为 1 时把 v 异或到累加器 z ；随后 v 右移一位，若移出最低位 1，则异或 $0xe1$ 完成模多项式约简。该实现清楚展示 GHASH 数学定义，但每个 128 bit 乘法需要 128 轮，因此性能较低。

4.12 PCLMULQDQ 优化 GHASH

Listing 13: 使用无进位乘法生成四个部分积

```
1 const __m128i z00 = _mm_clmulepi64_si128(a, b, 0x00);
2 const __m128i z01 = _mm_clmulepi64_si128(a, b, 0x01);
3 const __m128i z10 = _mm_clmulepi64_si128(a, b, 0x10);
4 const __m128i z11 = _mm_clmulepi64_si128(a, b, 0x11);
5 const __m128i middle = _mm_xor_si128(z01, z10);
```

两个 128 bit 多项式分别拆成高低 64 bit。四条 PCLMULQDQ 产生低低、低高、高低和高高部分积，交叉项异或后拼接为 256 bit 乘积，再依据 GCM 不可约多项式约简。与逐比特实现相比，硬件无进位乘法显著降低认证开销。

4.13 GCM 认证数据组织

Listing 14: AAD、密文与长度块的 GHASH 顺序

```
1 ghash(ctx->hash_subkey, aad, aad_len, s);
2 ghash(ctx->hash_subkey, ciphertext, text_len, s);
3 store_be64(lengths, (uint64_t)aad_len * 8U);
4 store_be64(lengths + 8, (uint64_t)text_len * 8U);
5 ghash(ctx->hash_subkey, lengths, 16, s);
```

GCM 认证的对象不是明文，而是 AAD、密文以及二者的比特长度。长度块采用两个 64 bit 大端整数。严格遵循该顺序能够兼容 NIST 测试向量，并防止不同长度消息在简单零填充下产生歧义。

4.14 GCM 标签验证与失败处理

Listing 15: 先验证标签，再输出明文

```
1 unsigned diff = 0;
2 for (unsigned i = 0; i < tag_len; ++i) {
3     diff |= expected_tag[i] ^ tag[i];
4 }
5 if (diff != 0U) {
6     if (plaintext != NULL && text_len != 0U) {
7         memset(plaintext, 0, text_len);
8     }
9     return -1;
10 }
```

比较过程不在发现第一个不同字节时提前退出，而是累计所有差异，降低简单时序泄露。若标签错误，函数返回失败并清零明文缓冲区，避免调用者误用未经认证的数据。实验专门修改标签中的一个比特，验证拒绝解密逻辑。

4.15 XTS tweak 更新

Listing 16: $GF(2^{128})$ 中乘 α

```
1 static void gf_mul_alpha(uint8_t tweak[16]) {
2     unsigned carry = 0;
3     for (unsigned i = 0; i < 16; ++i) {
4         const unsigned next = tweak[i] >> 7;
5         tweak[i] = (uint8_t)((tweak[i] << 1) | carry);
6         carry = next;
7     }
8     if (carry != 0U) {
9         tweak[0] ^= 0x87U;
10    }
11 }
```

XTS 标准把 tweak 看作小端多项式表示，因此循环从低地址字节向高地址传播进位。最高位溢出时异或 0x87 完成约简。每处理一个分组更新一次 tweak，使相同明文出现在不同分组位置时产生不同密文。

4.16 XTS 分组加密

Listing 17: XEX 结构的单分组计算

```
1 for (unsigned i = 0; i < 16; ++i) {  
2     x[i] = in[i] ^ tweak[i];  
3 }  
4 encrypt_block(&ctx->data_key, x, y);  
5 for (unsigned i = 0; i < 16; ++i) {  
6     out[i] = y[i] ^ tweak[i];  
7 }
```

明文先与位置相关 tweak 异或，再使用数据密钥 K_1 执行 AES，最后再次异或同一 tweak。这是 XEX 结构。tweak 本身由第二个密钥 K_2 加密数据单元编号得到，因此两个密钥承担不同功能，不能互换为单密钥实现。

4.17 SM4 非线性与线性变换

Listing 18: SM4 轮函数中的复合变换 T

```
1 uint32_t sm4_t(uint32_t x) {  
2     const uint32_t b = sm4_tau(x);  
3     return b ^ sm4_rotl32(b, 2)  
4         ^ sm4_rotl32(b, 10)  
5         ^ sm4_rotl32(b, 18)  
6         ^ sm4_rotl32(b, 24);  
7 }
```

sm4_tau 对 32 bit 输入的 4 个字节分别查 SM4 S 盒，随后按照标准执行四个循环移位并异或。该函数在基础 32 轮中被重复调用，是 SM4 标量实现的主要计算热点。

4.18 SM4 基础 32 轮

Listing 19: SM4 基础轮迭代

```
1 for (unsigned i = 0; i < 32; ++i) {  
2     const uint32_t n =  
3         x0 ^ sm4_t(x1 ^ x2 ^ x3 ^ rk[i]);  
4     x0 = x1;  
5     x1 = x2;  
6     x2 = x3;  
7     x3 = n;  
8 }  
9 sm4_store_be32(out, x3);  
10 sm4_store_be32(out + 4, x2);  
11 sm4_store_be32(out + 8, x1);  
12 sm4_store_be32(out + 12, x0);
```

四个状态字以滑动窗口方式更新。轮后按 x_3, x_2, x_1, x_0 反序输出。加密与解密共享同一轮函数，只由传入的轮密钥顺序决定方向，这减少了重复代码并降低两套实现不一致的风险。

4.19 SM4 T-table 构造

Listing 20: 将 SM4 S 盒与线性变换合并

```
1 for (unsigned x = 0; x < 256; ++x) {  
2     SM4_T0[x] = linear((uint32_t)SM4_SBOX[x] << 24);  
3     SM4_T1[x] = linear((uint32_t)SM4_SBOX[x] << 16);  
4     SM4_T2[x] = linear((uint32_t)SM4_SBOX[x] << 8);  
5     SM4_T3[x] = linear((uint32_t)SM4_SBOX[x]);  
6 }
```

每张表表示一个 S 盒输出字节放置在不同字节位置后，经过完整线性变换 L 产生的 32 bit 贡献。由于 L 是线性映射，四个字节的贡献可以分别计算后异或，从而把 4 次 S 盒和多个循环移位替换为 4 次查表。

4.20 SM4 AVX2 八路并行查表

Listing 21: VPGATHERDD 并行读取 SM4 四张 T 表

```
1 const __m256i a = _mm256_i32gather_epi32(  
2     (const int *)SM4_T0, i0, 4);  
3 const __m256i b = _mm256_i32gather_epi32(  
4     (const int *)SM4_T1, i1, 4);  
5 const __m256i c = _mm256_i32gather_epi32(  
6     (const int *)SM4_T2, i2, 4);  
7 const __m256i d = _mm256_i32gather_epi32(  
8     (const int *)SM4_T3, i3, 4);  
9 return _mm256_xor_si256(  
10     _mm256_xor_si256(a, b),  
11     _mm256_xor_si256(c, d));
```

i_0 至 i_3 分别保存 8 个 lane 中对应字节的表索引。Gather 指令从非连续地址并行加载 8 个 32 bit 表项，随后向量异或得到 8 个分组的 T 变换结果。尽管 Gather 的延迟高于连续加载，但跨 8 分组并行仍能显著提高总体吞吐量。

4.21 SM4-CTR 八路处理

Listing 22: SM4-CTR 的 AVX2 主循环与尾部回退

```
1 while (len >= 128U) {  
2     for (unsigned b = 0; b < 8; ++b) {  
3         memcpy(counters + 16U * b, counter, 16);  
4         increment_be(counter);  
5     }  
6     sm4_encrypt8_avx2(ctx, counters, stream);  
7     for (unsigned i = 0; i < 128; ++i) {  
8         out[i] = (uint8_t)(in[i] ^ stream[i]);  
9     }  
10    in += 128;  
11    out += 128;
```

```
12     len -= 128;
13 }
14 if (len != 0U) {
15     serial(ctx, counter, in, out, len,
16           sm4_encrypt_ttable);
17 }
```

主循环结构与 AES-CTR VAES 版本一致，体现了工作模式层与底层分组密码后端的分离。只有批量加密函数不同，计数器生成、异或和尾部分支可以复用相同设计思想。

4.22 GIFT-64 基础轮函数

Listing 23: GIFT-64 的 SubCells、PermBits 与轮掩码注入

```
1 for (unsigned r = 0; r < GIFT64_ROUNDS; ++r) {
2     s = sub_cells(s, GIFT_S);
3     s = perm_bits(s, GIFT_P);
4     s ^= ctx->round_masks[r];
5 }
```

状态使用一个 64 bit 整数保存，最低到最高的 16 个半字节依次经过 4 bit S 盒。随后 `perm_bits` 按照固定表 `GIFT_P` 移动每一个比特，最后异或当前轮预先生成的轮掩码。该写法严格对应算法定义，结构直观，适合作为正确性基准；其主要开销来自每轮 16 次 S 盒处理和 64 次逐比特置换。

4.23 GIFT-64 SP-table 构造

Listing 24: 合并 GIFT S 盒与比特置换

```
1 for (unsigned pos = 0; pos < 16; ++pos) {
2     for (unsigned x = 0; x < 16; ++x) {
3         uint64_t y = 0;
4         const uint8_t s = GIFT_S[x];
5         for (unsigned b = 0; b < 4; ++b) {
6             y |= (uint64_t)((s >> b) & 1U)
7                 << GIFT_P[4U * pos + b];
8         }
9         SP_TABLE[pos][x] = y;
10    }
11 }
```

对某个位置 `pos` 和输入半字节 `x`，表项已经包含 S 盒输出 4 个比特在完整 64 bit 置换结果中的贡献。实际轮函数只需从 16 张小表中各取一个表项并异或，不再执行逐比特置换。与 AES 的 4 张 256 项 T 表相比，GIFT 表的每张表仅 16 项，更符合轻量密码的小 S 盒特征。

4.24 GIFT-64 AVX2 八分组并行

Listing 25: 使用 64 bit Gather 并行完成 GIFT SP-table 轮

```
1 for (unsigned i = 0; i < 16; ++i) {
2     __m256i idx = _mm256_and_si256(
```

```
3     _mm256_srli_epi64(s, 4U * i), nibble_mask);
4     __m256i v = _mm256_i64gather_epi64(
5         (const long long *)SP_TABLE[i], idx, 8);
6     y = _mm256_xor_si256(y, v);
7 }
8 return _mm256_xor_si256(
9     y, _mm256_set1_epi64x((long long)mask));
```

一个 `__m256i` 包含 4 个 64 bit GIFT 状态。函数同时维护两个向量，因此一次处理 8 个分组。循环先从每个 lane 取出同一位置的半字节，再用 `VPGATHERQ` 读取对应 64 bit SP-table 项。轮掩码广播到所有 lane 后统一异或。该实现保留各分组内部的 28 轮依赖，但通过 8 条独立状态链提高指令级并行度。

4.25 TWINE 基础 F 层与轮置换

Listing 26: TWINE 一轮的八个 F 函数

```
1 static void f_layer(uint8_t x[16],
2                     const uint8_t rk[8]) {
3     for (unsigned j = 0; j < 8; ++j) {
4         x[2*j + 1] ^= S[x[2*j] ^ rk[j]];
5     }
6 }
7
8 for (unsigned r = 0; r < 35; ++r) {
9     f_layer(x, ctx->round_keys[r]);
10    permute(x, PI);
11 }
12 f_layer(x, ctx->round_keys[35]);
```

TWINE 每个 F 函数只读取一个偶数半字节和一个 4 bit 轮密钥，并把 S 盒输出异或到相邻奇数半字节。前 35 轮之后应用固定置换，末轮不置换。代码中的循环边界直接体现 36 轮结构，避免把末轮错误地与普通轮完全相同处理。

4.26 TWINE-80 与 TWINE-128 密钥扩展

Listing 27: TWINE 密钥扩展中的非线性更新与轮常量注入

```
1 w[1] ^= S[w[0]];
2 w[4] ^= S[w[16]];
3 w[7] ^= (c >> 3) & 7U;
4 w[19] ^= c & 7U;
5
6 /* TWINE-128 additionally updates: */
7 w[23] ^= S[w[30]];
```

两种密钥调度都把密钥拆成 4 bit 半字节，并在每轮执行 S 盒更新、轮常量注入和半字节重排。TWINE-128 比 TWINE-80 多一处 S 盒反馈，同时从 32 个半字节中的不同位置抽取 8 个轮密钥半字节。初始化阶段把 36 组轮密钥全部保存到上下文，使数据加密路径只保留 F 层与置换。

4.27 TWINE AVX2 寄存器内 Shuffle

Listing 28: VPSHUFB 实现 TWINE S 盒和固定置换

```
1 __m256i z = _mm256_xor_si256(  
2     x, make_key_vec(rk));  
3 z = _mm256_shuffle_epi8(tab, z);  
4 x = _mm256_xor_si256(  
5     x, _mm256_shuffle_epi8(z, move_mask));  
6 x = _mm256_shuffle_epi8(x, perm_mask);
```

`tab` 在每个 128 bit lane 中保存完整 16 项 S 盒，半字节值可直接作为 VPSHUFB 索引。`move_mask` 把 8 个 S 盒结果移动到奇数位置并与原状态异或，`perm_mask` 完成 π 置换。整个 F 层不访问秘密相关内存地址。四个 AVX2 状态向量共容纳 8 个分组，80 bit 与 128 bit 版本共享同一批量轮函数，只传入不同的预计算轮密钥。

4.28 统一编译与运行脚本

Listing 29: 一键实验流程

```
1 make clean  
2 make -j"$(nproc)"  
3 make test  
4 make benchmark  
5 MPLBACKEND=Agg make plot
```

脚本先进行全量重编译，再用较小数据量执行全部已知答案测试，随后运行正式吞吐率测试。绘图显式使用 Agg 后端，使 WSL 无需 X11 或 Qt 即可生成 PNG。这样既避免旧目标文件影响结果，也保证正确性测试先于性能测试执行。

5 正确性验证与实验步骤

5.1 标准测试向量

本实验使用公开标准与固定向量验证实现：

1. AES-128 使用 FIPS-197 向量：密钥 000102...0f、明文 001122...ff，期望密文 69c4e0d86a7b0430d8cdb78070b4c55a；
2. AES-CTR 使用 NIST SP 800-38A F.5.1 向量，验证多分组计数器与密钥流；
3. AES-GCM 验证空明文、单个全零分组、带 AAD 与非整分组消息、一般长度 IV；
4. SM4 使用 GB/T 32907 示例：密钥与明文均为 0123456789abcdeffedcba9876543210，期望密文为 681edf34d206965e86b3e94f536e4246；
5. XTS 使用与 OpenSSL/cryptography 交叉核对的完整分组和 CTS 固定向量；
6. GIFT-64 使用 3 组公开向量，其中全零密钥、全零明文的期望密文为 f62bc3ef34f775ac，并同时验证基础、SP-table、解密和 AVX2 八路实现；

7. TWINE 使用原算法向量:明文 0123456789abcdef, TWINE-80 期望密文 7c1f0f80b1df9c28, TWINE-128 期望密文 979ff9b379b5a9b8;
8. 各优化后端同时与基础实现比较, 验证 8 路并行、原地操作和尾分组结果一致。

5.2 鲁棒性测试

除标准向量外, 测试程序还覆盖以下边界条件:

- CTR 输入长度不是 16 字节整数倍;
- CTR 计数器低位从 0xff 溢出并向高位进位;
- VAES/AVX2 至少进入一次 128 字节主循环, 再处理尾部;
- GCM 标签被修改一个比特时必须拒绝解密;
- GCM 拒绝解密后清零输出缓冲区;
- GCM 使用非 96 bit IV 时正确通过 GHASH 生成 J_0 ;
- XTS 数据长度包含不完整尾分组时执行 ciphertext stealing;
- CTR、GCM 和 XTS 支持输入输出缓冲区相同的原地操作;
- GIFT 和 TWINE 把同一标准明文复制到 8 个 lane, 验证 AVX2 批量输出逐分组等于基础实现。

5.3 实际运行步骤

在 WSL 中将工程解压后执行:

Listing 30: 编译并运行完整实验

```
1 chmod +x run_all.sh
2 ./run_all.sh
```

脚本产生 6 个分类 CSV 和 1 个合并 CSV:

Listing 31: 性能结果文件

```
1 results/aes_benchmark.csv
2 results/ctr_benchmark.csv
3 results/gcm_benchmark.csv
4 results/xts_benchmark.csv
5 results/sm4_benchmark.csv
6 results/lightweight_benchmark.csv
7 results/all_benchmarks.csv
8 results/performance.png
```

5.4 正确性测试结果

表 2: 各算法与工作模式测试结果

模块	测试内容	结果
AES	Basic、T-table、Shuffle、AES-NI、VAES 加密及三种解密	PASS
AES-CTR	五种后端、部分分组、计数器进位、VAES 主循环与尾部	PASS
AES-GCM	GHASH 一致性、AAD、一般 IV、Tag 篡改、原地与尾分组	PASS
AES-XTS	完整分组、AES-NI、VAES、CTS 尾分组	PASS
SM4	基础、T-table、AVX2 八路、国标向量	PASS
SM4-CTR	AVX2 主循环与尾分组	PASS
GIFT-64	3 组公开向量、基础加解密、SP-table、AVX2 八路	PASS
TWINE	TWINE-80/128 公开向量、基础加解密、AVX2 八路	PASS
Sanitizer	AddressSanitizer 与 UndefinedBehaviorSanitizer	PASS

所有核心正确性测试均通过，说明优化版本与基础算法保持等价。编译阶段仅曾出现测试代码忽略 `system()` 返回值的警告，后续版本已修复；绘图脚本也改为 Agg 后端，解决 WSL 缺少 Qt/X11 时的 `xcb` 错误。这些问题均不影响密码算法结果。

6 性能结果与分析

6.1 性能测试方法

性能测试使用单调时钟记录批量加密耗时，以 MiB/s 表示吞吐率。AES 和 SM4 分组测试对同一密钥上下文重复处理数据；CTR、GCM 和 XTS 测试使用大缓冲区，减少启动、内存分配和计时函数对结果的影响。正式结果中 AES、GCM 和 SM4 使用 64 MiB，CTR 和 XTS 使用 128 MiB；考虑到 GIFT 基础逐比特置换速度较低，轻量密码统一使用 8 MiB 数据量。数据均来自用户的 Intel Core i7-13700H WSL 环境。

需要说明的是，基础版、T-table、Shuffle 和 AES-NI 单分组函数每次处理 16 字节，而 VAES 函数每次处理 8 个分组。VAES 结果代表可并行批量负载的最大吞吐率，不能直接解释为单条消息或单个分组的延迟缩短相同比例。

6.2 AES 分组密码性能

表 3: AES-128 不同实现的吞吐率与加速比

实现	吞吐率 (MiB/s)	相对基础版
AES-128 basic	58.55	1.00×
AES-128 T-table	277.15	4.73×
AES-128 shuffle	128.42	2.19×
AES-128 AES-NI	948.97	16.21×
AES-128 VAES x8	7867.71	134.38×

T-table 达到 277.15 MiB/s, 是基础版的 4.73 倍, 说明把 S 盒与 MixColumns 合并后能够显著减少有限域乘法。Shuffle 版本为 128.42 MiB/s, 约为基础版 2.19 倍, 证明字节重排和向量化 MixColumns 有效, 但普通 S 盒查表仍是主要瓶颈。

AES-NI 达到 948.97 MiB/s, 是基础版的 16.21 倍。专用轮指令同时完成非线性替换、置换、列混合和轮密钥加, 避免了软件指令数量和秘密相关 T-table 访存。VAES 八路达到 7867.71 MiB/s, 说明在批量独立分组场景下, 多 lane 和多依赖链能够充分利用处理器吞吐能力。

6.3 AES-CTR 性能

表 4: AES-CTR 不同实现的吞吐率与加速比

实现	吞吐率 (MiB/s)	相对基础版
AES-CTR basic	55.45	1.00×
AES-CTR T-table	265.02	4.78×
AES-CTR shuffle	126.36	2.28×
AES-CTR AES-NI	910.06	16.41×
AES-CTR VAES x8	3870.23	69.79×

CTR 各版本的相对趋势与 AES 分组测试一致。基础 CTR 比基础 AES 略慢, 是因为还包含计数器生成和明密文异或。VAES CTR 达到 3870.23 MiB/s, 低于直接 VAES 分组测试的 7867.71 MiB/s, 原因包括计数器复制、128 bit 递增、密钥流写回和输入异或开销。尽管如此, CTR 天然不存在分组间依赖, 仍是最能发挥 VAES 并行能力的工作模式之一。

6.4 AES-GCM 性能

表 5: AES-GCM 不同实现的吞吐率与加速比

实现	吞吐率 (MiB/s)	相对基础版
AES-GCM basic	8.84	1.00×
AES-NI + PCLMULQDQ	393.69	44.53×
VAES x8 + PCLMULQDQ	527.89	59.71×

基础 GCM 只有 8.84 MiB/s, 明显低于基础 CTR, 说明逐比特 GHASH 是主要瓶颈。AES-NI 与 PCLMULQDQ 组合后提升至 393.69 MiB/s, 达到 44.53 倍加速; 这不仅来自 AES 轮函数优化, 更重要的是硬件无进位乘法替代了每个认证块的 128 轮位操作。

VAES+PCLMULQDQ 达到 527.89 MiB/s, 较 AES-NI+PCLMULQDQ 进一步提升, 但没有像纯 CTR 那样成倍增长。这是因为 GHASH 满足 $X_i = (X_{i-1} \oplus B_i)H$, 当前块依赖前一块认证状态, 串行依赖限制了整体并行度。实验结果清楚表明, 优化组合工作模式时不能只关注底层分组密码, 还必须分析模式自身的数据依赖。

6.5 AES-XTS 性能

表 6: AES-XTS 不同实现的吞吐率与加速比

实现	吞吐率 (MiB/s)	相对基础版
AES-XTS basic	53.47	1.00×
AES-XTS AES-NI	411.32	7.69×
AES-XTS VAES x8	698.44	13.06×

XTS 除了 AES 加密外, 每个分组还需执行两次 tweak 异或和一次 $GF(2^{128})$ 乘 α , 因此 AES-NI 版本为 411.32 MiB/s, 低于 CTR AES-NI。VAES 版本达到 698.44 MiB/s, 但 tweak 必须按顺序生成, 且输入输出需要在 AES 前后异或各自 tweak, 限制了宽向量的收益。该结果符合 XTS 的结构特点。

6.6 SM4 与 SM4-CTR 性能

表 7: SM4 及 SM4-CTR 吞吐率与加速比

实现	吞吐率 (MiB/s)	相对各自基础版
SM4 basic	79.57	1.00×
SM4 T-table	116.06	1.46×
SM4 AVX2 x8	225.44	2.83×
SM4-CTR basic	74.33	1.00×
SM4-CTR T-table	116.84	1.57×
SM4-CTR AVX2 x8	259.04	3.48×

SM4 T-table 仅获得 1.46 倍加速，低于 AES T-table 的 4.73 倍。这是因为 SM4 基础轮本身以 32 bit 字为核心，线性变换主要由循环移位和异或组成，标量成本没有 AES 逐列有限域乘法那么高；同时 32 轮中的状态依赖仍然存在。

AVX2 八路版本达到 225.44 MiB/s，是基础版的 2.83 倍。加速没有达到理论 8 倍，一方面是 Gather 查表延迟较高，另一方面是数据需要在“按分组存储”和“按状态字跨分组向量化”之间整理。SM4-CTR AVX2 达到 259.04 MiB/s，相对基础 SM4-CTR 为 3.48 倍，说明批量计数器提供了稳定的 8 路独立输入，能够更充分利用 AVX2 后端。

6.7 GIFT 与 TWINE 轻量密码性能

表 8: GIFT-64 与 TWINE 吞吐率及加速比

实现	吞吐率 (MiB/s)	相对各自基础版
GIFT-64 basic	6.43	1.00×
GIFT-64 T-table	54.29	8.44×
GIFT-64 AVX2 x8	82.55	12.83×
TWINE-80 basic	12.82	1.00×
TWINE-80 AVX2 x8	65.21	5.09×
TWINE-128 basic	12.70	1.00×
TWINE-128 AVX2 x8	64.01	5.04×

GIFT-64 基础版只有 6.43 MiB/s，主要瓶颈不是 4 bit S 盒，而是 28 轮中逐比特执行 64 bit 置换。SP-table 把 S 盒和 PermBits 合并后达到 54.29 MiB/s，取得 8.44 倍加速，说明消除逐比特循环是 GIFT 软件优化的关键。AVX2 八路进一步达到 82.55 MiB/s，相对基础版提升 12.83 倍；它相对 SP-table 仍有约 1.52 倍提升，收益受到 64 bit Gather 延迟和每轮 16 次 Gather 操作限制。

TWINE-80 与 TWINE-128 基础版分别为 12.82 MiB/s 和 12.70 MiB/s，数值非常接近，因为

密钥扩展在初始化阶段完成，正式加密路径都执行相同的 36 轮 F 层。AVX2 八路分别达到 65.21 MiB/s 和 64.01 MiB/s，约为基础版的 5.09 倍和 5.04 倍。其加速来自 VPSHUFB 同时完成多个 4 bit S 盒查找和固定半字节置换；未达到理论 8 倍的原因包括输入输出的半字节展开与压缩、四个向量状态的维护，以及轮密钥向量构造开销。

6.8 总体性能图

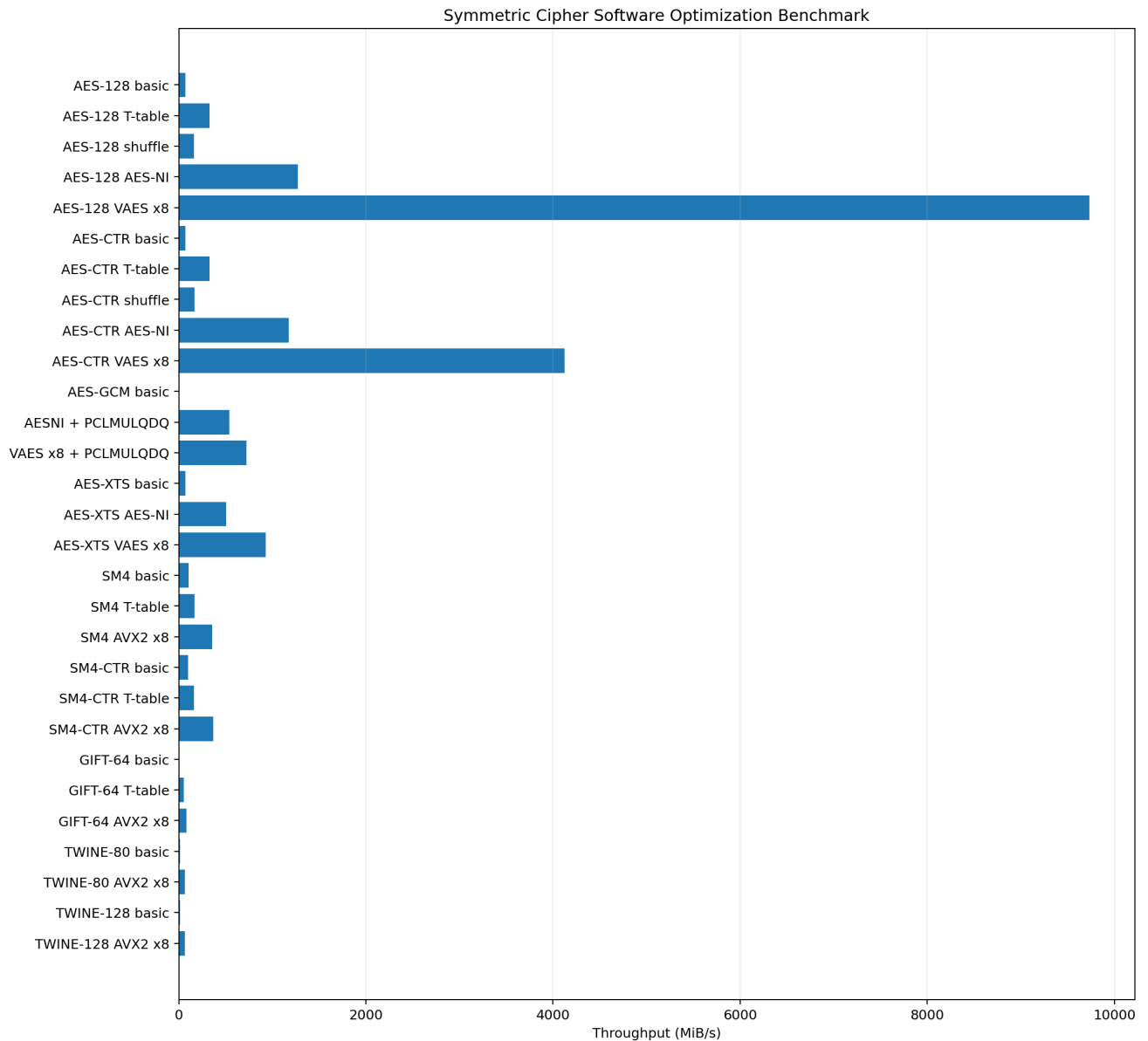


图 1: AES、SM4、GIFT、TWINE 及 CTR/GCM/XTS 各实现吞吐率对比

图1采用对数或分组横向展示时，可以同时观察低吞吐率基础 GCM 与高吞吐率 VAES，避免大数值使其他柱形不可见。总体上，专用指令优化效果最强；T-table 和 Shuffle 属于在没有专用密码指令时仍有价值的软件优化；工作模式的最终性能则由底层密码速度、模式依赖、计数器/tweak 处理和内存异或共同决定。

6.9 结果有效性分析

实验结果在趋势上具有一致性：

1. AES 分组与 AES-CTR 中，Basic、Shuffle、T-table、AES-NI、VAES 的性能排序基本一致；
2. GCM 基础版因逐比特 GHASH 显著慢于 CTR，使用 PCLMULQDQ 后出现数量级提升；
3. XTS 的专用指令加速明显，但受 tweak 处理影响，低于 CTR；
4. SM4 T-table 提升有限，而跨分组 AVX2 并行进一步提高吞吐率；
5. GIFT 的 SP-table 消除了逐比特置换热点，TWINE 的寄存器内 shuffle 避免了逐半字节标量 S 盒，两者的 AVX2 八路均继续提高吞吐率；
6. 所有版本输出 checksum 一致或通过固定向量，排除了“少算轮数换取速度”的错误优化。

短测试与正式测试之间存在少量波动，这是 CPU 动态频率、WSL 调度、缓存热度和计时长度造成的正常现象。报告采用 64/128 MiB 正式数据，而不是 2 MiB 快速测试数据，以降低固定开销对结果的影响。

7 安全性与工程讨论

7.1 T-table 缓存侧信道

T-table 索引来自加密中间状态，而中间状态与密钥相关。攻击者若能观察缓存命中、访问时延或共享缓存集合，就可能推测表索引并进一步恢复密钥。因此 T-table 虽然提高了纯软件吞吐率，却不是常数时间实现。本实验保留该方法是为了满足作业要求和比较优化原理，实际安全软件应优先使用 AES-NI、VAES 或经过严格设计的 bitslicing 方案。

SM4 T-table 和 AVX2 Gather 同样存在秘密相关访存。向量化只改变一次处理的分组数量，并不会自动消除侧信道。在需要抗侧信道的 SM4 实现中，可考虑使用硬件 SM4 指令、逻辑电路化 S 盒或经过验证的常数时间实现。

GIFT 的 SP-table 及 VPGATHERQ 索引同样由秘密中间状态决定，因此存在缓存与 Gather 时延侧信道风险。TWINE 的 AVX2 版本则把 16 项 S 盒完整放入向量寄存器，通过 VPSHUF 完成寄存器内选择，内存访问地址与秘密无关，具有更好的常数时间实现特征。

7.2 CTR 的 Nonce/计数器复用风险

CTR 安全性要求同一密钥下不得重复使用相同初始计数器。若两个消息使用同一密钥流，则

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

攻击者可消除密钥流并利用明文冗余恢复内容。代码正确递增计数器只能保证单次调用内部不重复，系统层仍必须负责分配唯一 Nonce 并防止计数器空间溢出。

7.3 GCM 的 IV 唯一性与认证顺序

GCM 对 IV 复用尤其敏感。同一密钥和 IV 复用不仅重复 CTR 密钥流，还会破坏 GHASH 认证安全。本实现支持 96 bit 和一般长度 IV，但调用者必须保证唯一性。解密时应先验证 Tag，再把明文交给上层。本工程在标签失败时清零输出，体现“未认证数据不可使用”的原则。

7.4 XTS 的安全边界

XTS 用于存储介质的扇区级机密性保护，通过 tweak 避免相同明文块在不同位置产生相同密文。但 XTS 不提供完整性和来源认证，攻击者仍可能翻转或重排密文块。因此涉及主动攻击时，应在 XTS 之外增加完整性保护或采用具备认证能力的存储方案。

7.5 性能测试的公平性

为了公平比较，各版本使用相同密钥长度、相同算法轮数和相同输入规模，并在测试前先验证正确性。VAES 和 AVX2 属于批量接口，其优势依赖足够多独立分组；对于只有单个 16 字节分组的低延迟请求，AES-NI 更具有代表性。报告同时给出吞吐率而不声称所有场景都能获得相同加速。

8 实验中遇到的问题与解决方法

8.1 脚本执行权限问题

首次运行 `./run_all.sh` 时出现 `Permission denied`，原因是 ZIP 解压后脚本未保留可执行位。解决方法为：

```
1 chmod +x run_all.sh
2 ./run_all.sh
```

也可以使用 `bash run_all.sh` 直接由解释器执行。最终提交版对脚本设置了可执行权限，并在 README 中写明处理方式。

8.2 WSL 绘图后端问题

算法测试和 CSV 生成全部成功后，Matplotlib 曾尝试加载 Qt 的 `xcb` 插件。WSL 终端没有图形显示环境，导致绘图阶段终止。解决方法是在绘图脚本开头固定无界面后端：

```
1 import matplotlib
2 matplotlib.use("Agg")
3 import matplotlib.pyplot as plt
```

或者执行 `MPLBACKEND=Agg make plot`。Agg 直接把图像渲染为 PNG，不需要 Qt 或 X11，适合服务器和 WSL 环境。

8.3 优化正确性问题

T-table、Shuffle 和 VAES 优化容易出现状态字节序、ShiftRows 索引、末轮处理和轮密钥顺序错误。解决方法不是只观察最终程序是否运行，而是让每个后端分别通过同一 FIPS 向量，并比较单分组输出。CTR/GCM/XTS 又增加计数器、长度块和 tweak 字节序，因此分别增加专门向量和边界测试。

8.4 尾分组与原地操作

CTR 和 GCM 可处理任意长度，XTS 则需要 CTS。实现中始终在覆盖输入前完成当前分组所需数据读取，并为尾分组使用局部临时缓冲区。测试程序使用输入输出指针相同的场景验证原地解密，避免只在独立缓冲区条件下正确。

8.5 轻量密码的半字节与比特映射

GIFT 与 TWINE 都以 4 bit 半字节为核心, 但输入输出按字节存储, 最容易出现的问题是半字节编号、大小端和置换方向不一致。GIFT 还涉及“源比特到目标比特”的 PermBits 映射, 解密必须使用严格的逆映射; TWINE 则要求前 35 轮置换、末轮不置换。实现中把字节-半字节转换、正逆置换和标准向量分别封装测试, 再让 SP-table 和 AVX2 后端与基础实现交叉比较, 从而避免仅凭加解密互逆而掩盖同方向错误。

9 实验总结

本实验完整实现了 AES-128、SM4、GIFT-64 以及 TWINE-80/128, 并从基础软件、T-table 或 SP-table、Shuffle/SIMD、AES-NI、VAES、AVX2 及 PCLMULQDQ 多个层次进行优化; 进一步将优化后的 AES 与 SM4 用于 CTR、GCM 和 XTS 模式。工程不仅包含加密, 也实现必要的解密、认证验证、计数器与 tweak 更新、一般长度输入和 ciphertext stealing。

正确性方面, AES 通过 FIPS-197 向量, CTR 和 GCM 通过 NIST 向量, SM4 通过 GB/T 32907 示例, XTS 结果与成熟实现交叉核对; GIFT-64 通过 3 组公开向量, TWINE-80 和 TWINE-128 均通过原算法固定向量; 所有优化后端、尾分组、原地操作和标签篡改测试均通过。性能方面, AES T-table 相对基础版提升 4.73 倍, AES-NI 提升 16.21 倍, VAES 八路批量吞吐率达到 7867.71 MiB/s。AES-CTR VAES 达到 3870.23 MiB/s; GCM 在 PCLMULQDQ 优化后从 8.84 MiB/s 提高到 393.69–527.89 MiB/s; SM4 AVX2 和 SM4-CTR AVX2 分别达到 2.83 倍和 3.48 倍加速; GIFT-64 的 SP-table 与 AVX2 八路分别达到 8.44 倍和 12.83 倍, TWINE-80/128 的 AVX2 八路均获得约 5 倍加速。

实验说明, 对称密码优化不能简单理解为“使用更宽的寄存器”。首先应识别轮函数中的热点, 如 S 盒、有限域乘法、逐比特置换和固定半字节置换; 其次要根据工作模式判断分组间是否独立; 最后还要考虑访存、计数器、认证链、tweak 和尾部分组等额外开销。专用密码指令通常同时提供最高性能与更好的缓存侧信道特性, 而 T-table 虽然便于展示软件优化思想, 却应谨慎用于真实安全系统。

通过本次实验, 小组完成了从算法标准、C 语言实现、指令级优化到测试与性能分析的完整流程, 对“算法结构如何映射到处理器执行单元”以及“工作模式如何限制并行度”形成了更加具体的认识。

参考文献

- [1] National Institute of Standards and Technology. *FIPS 197: Advanced Encryption Standard (AES)*.
- [2] National Institute of Standards and Technology. *SP 800-38A: Recommendation for Block Cipher Modes of Operation*.
- [3] National Institute of Standards and Technology. *SP 800-38D: Recommendation for Galois/Counter Mode (GCM) and GMAC*.
- [4] National Institute of Standards and Technology. *SP 800-38E: Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices*.

- [5] 国家市场监督管理总局、国家标准化管理委员会. *GB/T 32907-2016 信息安全技术 SM4 分组密码算法*.
- [6] Subhadeep Banik et al. *GIFT: A Small Present*. CHES 2017.
- [7] Tomoyasu Suzaki et al. *TWINE: A Lightweight Block Cipher for Multiple Platforms*. SAC 2012.
- [8] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*.